IF

DOCUMENTATION TECHNIQUE · VERSION 1.1

ImmoFlow AI
Manuel du backend

Architecture, modules métier, modèle de sécurité, jeu de données de démonstration et référence complète de l'API.

85 POINTS D'ENTRÉE	**12** MODULES MÉTIER	**25** TABLES
5 RÔLES CLOISONNÉS	**26** TESTS AUTOMATISÉS	

FastAPI · PostgreSQL 16 · SQLAlchemy 2 · Redis · Docker — Cotonou, Bénin

Sommaire

01 Présentation de l'API 4

- Peupler la base 4
- Aperçu 4
- Stack technique 5
- Démarrage rapide 6
- Installation locale 6
- Variables d'environnement 6
- Architecture 7
- Modèle de sécurité 8
- Modules métier 9
- Documentation API 11

02 Architecture 7

- 1. Vue d'ensemble 15
- 2. Responsabilité de chaque couche 16
- 3. Gestion des transactions 17
- 4. Isolation multi-tenant 17
- 5. Le SecurityContext 17
- 6. Temps réel 18
- 7. Machines à états 18
- 8. Module IA — conception à double niveau 19
- 9. Observabilité 19
- 10. Choix techniques notables 20

03 Modules métier 21

- 1. Authentification 22
- 2. Agences 23
- 3. Agents 24
- 4. Propriétaires 26

5. Biens immobiliers 28

6. CRM Clients 30

7. Visites 31

8. Transactions 32

9. Documents 33

10. Messagerie temps réel 34

04 Jeu de démonstration 41

Utilisation 41

Volumes 41

Comptes créés 42

Ce que le script garantit 43

Reproductibilité 43

Suppression 44

Répartition des statuts 44

05 Installation 45

Option 1 — Docker (recommandé) 45

Option 2 — Installation locale 46

Vérification de l'installation 47

Dépannage 47

Mise à jour 47

PARTIE 1

Présentation de l'API

Ce que fait la plateforme, comment la démarrer, et comment les cinq rôles s'articulent.

Backend de la plateforme **ImmoFlow AI** — gestion immobilière multi-agences avec RBAC strict, messagerie temps réel et assistance IA.

> **Périmètre V1** — Aucun module de paiement, abonnement, facturation ou plan premium.

Peupler la base

```
cd backend
PYTHONPATH=. python scripts/seed_data.py                # 30 biens, 4 agences, 41 comptes
PYTHONPATH=. python scripts/seed_data.py --properties 200 --reset
```

Le script crée un jeu de données cohérent et vérifié : chaque bien appartient à un propriétaire de la même agence, chaque bien publié a une photo, un bien vendu quitte le catalogue. Voir `docs/DONNEES-DEMO.md`.

Aperçu

ImmoFlow AI couvre le cycle immobilier complet pour un réseau d'agences : publication de biens, gestion des propriétaires et des clients, visites, ventes, locations, documents, messagerie et assistance IA.

Cinq rôles, chacun avec un périmètre de données strictement délimité :

RÔLE	PÉRIMÈTRE
`SUPER_ADMIN`	Toute la plateforme
`AGENCY`	Uniquement les données de son agence
`AGENT`	Uniquement ce qui lui est attribué (biens, clients, propriétaires, visites, transactions)
`OWNER`	Uniquement ses biens et les interactions associées
`CLIENT`	Uniquement ses propres interactions + le catalogue publié

L'isolation multi-tenant n'est pas une simple vérification en début de handler : **chaque repository applique le périmètre directement dans la requête SQL** (`scoped_query`), de sorte qu'un oubli côté service ne peut pas exposer les données d'un autre tenant.

Stack technique

COMPOSANT	CHOIX
Langage	Python 3.12+
Framework	FastAPI 0.115
ORM	SQLAlchemy 2.0 (async, `Mapped[]`)
Base	PostgreSQL 16 (asyncpg)
Migrations	Alembic
Validation	Pydantic v2 + pydantic-settings
Auth	JWT (PyJWT) — access + refresh avec rotation
Mots de passe	bcrypt
Cache / Temps réel	Redis (Pub/Sub, rate limiting)
Temps réel	WebSocket natif FastAPI
Tâches	Celery (optionnel)
Conteneurs	Docker + Docker Compose
Tests	pytest + pytest-asyncio + httpx

Démarrage rapide

```
git clone <repo> && cd immoflow-ai-api
cp backend/.env.example backend/.env
# Générez une clé : openssl rand -hex 32  → renseignez SECRET_KEY
docker compose up --build
```

RESSOURCE	URL
Swagger UI	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc
OpenAPI JSON	http://localhost:8000/openapi.json
Santé	http://localhost:8000/health

Compte initial (modifiable via `.env`) : `admin@immoflow.ai` / `ChangeMe123!` **Changez ce mot de passe avant toute mise en ligne.**

Installation locale

Prérequis : Python 3.12+, PostgreSQL 16, Redis 7.

```
cd backend
python -m venv .venv && source .venv/bin/activate      # Windows : .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env                                    # puis ajustez les valeurs

createdb immoflow                                       # ou via psql
alembic upgrade head                                    # crée les 25 tables + 18 types énumérés
python -m app.database.init_db                          # Super Admin + types de biens

uvicorn app.main:app --reload
```

L'API démarre en mode **dégradé mais fonctionnel** si Redis est absent : le rate limiting et la diffusion inter-workers sont désactivés, le reste continue de fonctionner.

Variables d'environnement

Toutes les variables sont documentées dans `backend/.env.example`. Les plus structurantes :

VARIABLE	DÉFAUT	RÔLE
`ENVIRONMENT`	`development`	`development` / `staging` / `production` / `test`
`SECRET_KEY`	—	**Obligatoire en production.** `openssl rand -hex 32`
`ACCESS_TOKEN_EXPIRE_MINUTES`	`30`	Durée de vie de l'access token
`REFRESH_TOKEN_EXPIRE_DAYS`	`30`	Durée de vie du refresh token
`POSTGRES_*`	—	Connexion base de données
`REDIS_*`	—	Cache, Pub/Sub, rate limiting
`STORAGE_BACKEND`	`local`	`local` (disque) ou `s3`
`MAX_UPLOAD_SIZE_MB`	`25`	Taille maximale des fichiers
`AI_PROVIDER`	`none`	`none` / `openai` / `groq` / `anthropic`
`AI_API_KEY`	—	Clé du fournisseur LLM
`RATE_LIMIT_PER_MINUTE`	`120`	Quota global par identité
`SMTP_*`	—	Envoi d'emails (désactivé → journalisé)

Architecture

Architecture en couches, chacune avec une responsabilité unique :

```
Requête HTTP
   ↓
Middlewares        → identifiant de requête, chronométrage, en-têtes de sécurité, rate limiting
   ↓
API (routers)      → validation d'entrée/sortie, injection de dépendances, codes HTTP
   ↓
Dépendances        → authentification JWT → SecurityContext + Scope (périmètre RBAC)
   ↓
Services           → règles métier, transitions d'état, notifications, audit
   ↓
Repositories       → requêtes SQL avec application du périmètre (scoped_query)
   ↓
Modèles            → SQLAlchemy 2 (typage Mapped[])
   ↓
PostgreSQL
```

```
backend/
├── app/
│   ├── main.py                # application FastAPI, middlewares, cycle de vie
│   ├── core/
│   │   ├── config.py          # settings Pydantic
│   │   ├── security.py        # bcrypt, JWT, jetons opaques
│   │   ├── permissions.py     # SecurityContext + matrice RBAC
│   │   ├── exceptions.py      # exceptions métier + handlers globaux
│   │   ├── redis_client.py    # pool Redis partagé
│   │   └── logging_config.py  # logs JSON en production
│   ├── database/
│   │   ├── base.py            # Base déclarative + mixins
│   │   ├── session.py         # moteur async, sessions, sondes
│   │   ├── init_db.py         # données de référence
│   │   └── migrations/        # Alembic
│   ├── models/                # 25 tables SQLAlchemy
│   ├── schemas/               # contrats Pydantic v2
│   ├── repositories/          # accès données + isolation SQL
│   ├── services/              # logique métier
│   ├── api/
│   │   ├── deps.py            # dépendances (session, auth, RBAC, scope)
│   │   ├── router.py          # agrégation des routers
│   │   └── v1/                # 15 routers thématiques
│   ├── websocket/             # gestionnaire de connexions + pont Redis Pub/Sub
│   ├── middleware/            # contexte de requête, rate limiting
│   ├── worker/                # tâches Celery (optionnel)
│   └── utils/                 # pagination, slugs, fichiers
├── tests/
├── docker/
├── alembic.ini
├── requirements.txt
└── .env.example
```

Principes appliqués

- Dependency injection : aucune dépendance instanciée globalement dans les handlers ; la session, l'utilisateur et le périmètre arrivent par `Depends`.
- Séparation stricte : un router ne contient jamais de SQL, un repository ne contient jamais de règle métier.
- Transactions explicites : les services utilisent `flush()`, seul le router décide du `commit()` — une requête = une transaction.
- Défense en profondeur : RBAC en dépendance, puis en service, puis en SQL.

Modèle de sécurité

Authentification

- Mots de passe hachés avec bcrypt (12 tours par défaut).
- Access token JWT courte durée + refresh token long avec **rotation** : chaque usage révoque le précédent. Une réutilisation détectée révoque **toutes** les sessions de l'utilisateur (protection contre le vol de jeton).
- Refresh tokens et jetons de réinitialisation stockés **hachés (SHA-256)** en base.

- Comparaison à coût constant à la connexion pour éviter l'énumération de comptes.
- `/auth/forgot-password` répond identiquement que le compte existe ou non.

Autorisation

- Matrice de permissions par rôle (`core/permissions.py`), granularité `ressource:action`.
- `SecurityContext` transporte l'identité et le périmètre (agence, agent, propriétaire, client).
- Chaque repository expose `scoped_query(ctx)` qui traduit le périmètre en clauses `WHERE`.
- Les rôles `SUPER_ADMIN` et `AGENT` ne peuvent pas être obtenus par inscription publique.

Autres protections

- Rate limiting Redis avec quotas renforcés sur `/auth/*` (fail-open si Redis tombe).
- En-têtes de sécurité (`X-Content-Type-Options`, `X-Frame-Options`, HSTS en production).
- Validation stricte des uploads (type MIME sur liste blanche, taille, nom assaini).
- Protection contre la traversée de chemin dans le stockage local.
- Fichiers servis via l'API avec contrôle d'accès, jamais en accès direct.
- Journal d'audit sur toutes les actions sensibles (qui, quoi, quand, depuis quelle IP).

Modules métier

1. Authentification (`/auth`)

Inscription, connexion, refresh, déconnexion (session courante ou tous les appareils), mot de passe oublié/réinitialisation/changement, acceptation d'invitation, profil, avatar, liste des sessions actives.

2. Agences (`/agencies`)

Création, **validation par le Super Admin**, activation, suspension, informations, documents légaux, tableau de bord statistique. Cycle : `PENDING_VALIDATION → ACTIVE ⇄ SUSPENDED` (ou `REJECTED`).

3. Agents (`/agents`)

Création avec **invitation par email**, activation du compte via jeton, rattachement à l'agence, permissions fines (`can_manage_owners`, `can_publish_properties`), suspension, suppression.

4. Propriétaires (`/owners`) — deux workflows

Workflow 1 — invitation par l'agent

```
Agent crée (nom, téléphone, email) → invitation envoyée
   → le propriétaire complète son profil et dépose sa CIP
   → l'agence vérifie les documents → ACTIVE
```

Workflow 2 — inscription autonome

```
Le propriétaire s'inscrit auprès d'une agence active
   → PENDING_VERIFICATION → l'agence valide → ACTIVE
```

Statuts : `INVITED`, `PENDING_VERIFICATION`, `ACTIVE`, `REJECTED`, `SUSPENDED`. **Règle bloquante** : la validation est refusée tant qu'aucune pièce d'identité (catégorie `IDENTITY_CIP`) n'a été déposée.

5. Biens immobiliers (`/properties`, `/public/properties`)

CRUD complet, médias (URL ou téléversement), géolocalisation, recherche multicritères (prix, surface, chambres, ville, quartier, rayon géographique), tri, pagination, catalogue public sans authentification.

Statuts et transitions contrôlées :

```
DRAFT → AVAILABLE → RESERVED → SOLD | RENTED → ARCHIVED
```

Une transition interdite renvoie `409 business_rule_violation`. La publication exige : titre, prix, ville, description ≥ 40 caractères, ≥ 1 photo.

6. CRM Clients (`/clients`)

Fiche client, **rattachement à un agent après interaction** (`ClientAgent`, avec agent principal), favoris, historique de recherche, demandes, notes et budget.

7. Visites (`/visits`)

Demande (client ou agent), validation, refus motivé, reprogrammation, modification, annulation, clôture avec note et retour. Notification automatique du client, de l'agent et du propriétaire à chaque changement.

8. Transactions (`/sales`, `/rentals`, `/contracts`)

Vente : bien + client + agent + propriétaire, montant, taux et calcul automatique de commission. La finalisation bascule le bien en `SOLD` et le dépublie. **Location** : loyer, charges, caution, dates, durée. L'activation bascule le bien en `RENTED` ; la résiliation le remet en `AVAILABLE`. **Contrats** : acte de vente, bail, mandat — liés à un document signé.

9. Documents (`/documents`)

Téléversement contrôlé (liste blanche MIME, 25 Mo), rattachement polymorphe (propriétaire, agence, bien, contrat, utilisateur), catégories (CIP, contrat, mandat, titre de propriété, document légal), workflow de vérification, téléchargement authentifié avec journalisation.

10. Messagerie temps réel (`/conversations` , `/ws`)

Conversations `CLIENT ↔ AGENT` , `PROPRIÉTAIRE ↔ AGENT` , `AGENT ↔ AGENCE` . Messages texte, images, documents, vidéos. Historique paginé par curseur temporel, accusés de lecture, compteur de non-lus, indicateur de saisie. Diffusion multi-workers via **Redis Pub/Sub**.

11. Notifications (`/notifications`)

Invitation, validation de compte, nouveau message, visite, transaction, document. Triple canal : in-app (persisté), WebSocket (temps réel), email (optionnel).

12. IA immobilière (`/ai`)

ENDPOINT	FONCTION
`POST /ai/description`	Rédige une annonce professionnelle à partir des caractéristiques
`POST /ai/estimate`	Estime un prix : valeur conseillée, fourchette, confiance, explication
`POST /ai/assistant`	Recherche en langage naturel + conseil
`POST /ai/analyze-listing`	Score qualité, champs manquants, suggestions

Fournisseurs branchables : OpenAI, Groq, Anthropic (`AI_PROVIDER` + `AI_API_KEY`).

Sans clé LLM, les quatre fonctions restent opérationnelles grâce à des replis déterministes : génération par gabarits, estimation par médiane des biens comparables de l'agence (repli sur un barème régional au m²), analyse de requête par expressions régulières, audit d'annonce par règles. L'IA enrichit le résultat, elle n'en est pas la condition.

Exemple : `"Maison 4 chambres proche centre budget 60 millions"` → `{bedrooms: 4, city: "Cotonou", max_price: 60000000}` + biens correspondants.

Documentation API

Swagger interactif : **http://localhost:8000/docs** — 85 endpoints regroupés par module.

Séquence d'authentification

```
# 1. Connexion
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@immoflow.ai","password":"ChangeMe123!"}'

# 2. Utilisation
curl http://localhost:8000/api/v1/properties \
  -H "Authorization: Bearer <access_token>"

# 3. Renouvellement
curl -X POST http://localhost:8000/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"<refresh_token>"}'
```

Format de réponse paginée

```
{ "items": [...], "total": 42, "page": 1, "size": 20, "pages": 3 }
```

Format d'erreur

```
{ "error": { "code": "business_rule_violation",
             "message": "Transition de statut interdite : DRAFT → SOLD.",
             "request_id": "a1b2c3…" } }
```

Une **collection Postman** prête à l'emploi est fournie dans `postman/` (gestion automatique des jetons).

WebSocket

```
ws://localhost:8000/ws?token=<access_token>
```

SENS	ÉVÉNEMENTS
Entrants	`message.send`, `message.read`, `typing`, `ping`
Sortants	`connected`, `message.new`, `message.read`, `typing`, `notification.new`, `pong`, `error`

```
const ws = new WebSocket(`ws://localhost:8000/ws?token=${accessToken}`);

ws.onmessage = ({ data }) => {
  const { event, conversation_id, data: payload } = JSON.parse(data);
  if (event === "message.new") appendMessage(conversation_id, payload);
  if (event === "notification.new") showToast(payload.title);
};

ws.send(JSON.stringify({
  event: "message.send",
  conversation_id: conversationId,
  data: { type: "TEXT", content: "Bonjour" },
}));
```

Chaque message envoyé est persisté, diffusé aux participants connectés (via Redis Pub/Sub, donc à travers tous les workers) et génère une notification pour les absents.

Base de données et migrations

25 tables, 18 types énumérés PostgreSQL natifs.

Cœur : `users`, `agencies`, `agents`, `owners`, `clients`, `properties` **Rattachements** : `client_agents`, `client_favorites`, `client_search_history`, `client_requests`, `owner_invitations` **Activité** : `visits`, `sales`, `rentals`, `contracts`, `documents` **Communication** : `conversations`, `conversation_participants`, `messages`, `notifications` **Technique** : `property_types`, `property_images`, `audit_logs`, `refresh_tokens`, `password_reset_tokens`

```
alembic upgrade head                          # applique les migrations
alembic revision --autogenerate -m "message"  # crée une migration
alembic downgrade -1                          # revient d'un cran
alembic history                               # historique
```

Index composites sur les colonnes réellement filtrées : `(agency_id, status)` pour les biens, `(agency_id, scheduled_at)` et `(agent_id, status)` pour les visites, `(conversation_id, created_at)` pour les messages, `(user_id, is_read)` pour les notifications.

Tests

```
cd backend
createdb immoflow_test
POSTGRES_DB=immoflow_test pytest -v
```

La suite couvre l'authentification (rotation des jetons, rejet des mots de passe faibles, blocage de l'escalade de privilèges), le RBAC et l'**isolation multi-tenant**, le cycle de vie des biens et de la publication, les deux workflows propriétaire, les visites, les transactions, la messagerie et les quatre fonctions IA en mode repli.

Déploiement Docker

```
docker compose up --build -d     # API + PostgreSQL + Redis
docker compose logs -f api
docker compose --profile workers up -d   # ajoute le worker Celery
docker compose down -v           # arrêt + suppression des volumes
```

L'entrypoint attend PostgreSQL, applique les migrations, initialise les données de référence puis démarre Uvicorn (4 workers, `--proxy-headers`).

L'image de production est **multi-stage**, tourne sous un **utilisateur non-root** et expose un `HEALTHCHECK`.

Derrière un reverse proxy (Nginx, Traefik), pensez à transmettre `Upgrade` et `Connection` pour le WebSocket sur `/ws`.

Checklist production

- [ ] `SECRET_KEY` généré aléatoirement (`openssl rand -hex 32`) et stocké dans un coffre
- [ ] `ENVIRONMENT=production`, `DEBUG=false`
- [ ] Mot de passe du Super Admin changé après le premier démarrage
- [ ] `BACKEND_CORS_ORIGINS` et `ALLOWED_HOSTS` restreints aux domaines réels
- [ ] HTTPS terminé au niveau du reverse proxy (HSTS activé automatiquement)
- [ ] PostgreSQL et Redis non exposés publiquement, mot de passe Redis défini
- [ ] Sauvegardes automatiques de la base + test de restauration
- [ ] `STORAGE_BACKEND=s3` (ou volume persistant sauvegardé) pour les documents
- [ ] SMTP configuré (`SMTP_ENABLED=true`)
- [ ] Journaux JSON collectés vers un agrégateur, `request_id` conservé
- [ ] Sondes `/health/live` et `/health/ready` branchées sur l'orchestrateur
- [ ] Purge planifiée des jetons expirés (tâche Celery `tokens.purge_expired`)

Licence

Propriétaire — ImmoFlow AI.

PARTIE 2

Architecture

Organisation en couches, isolation multi-agences, transactions et modèle de données.

1. Vue d'ensemble

ImmoFlow AI est une API REST + WebSocket bâtie sur une architecture en couches inspirée de la clean architecture, adaptée au contexte d'une API FastAPI asynchrone.



2. Responsabilité de chaque couche

COUCHE	FAIT	NE FAIT JAMAIS
Router	Valide l'entrée/sortie, choisit le code HTTP, décide du `commit()`	Du SQL, des règles métier
Dépendance	Résout la session, l'utilisateur, le périmètre, applique les gardes RBAC	Des écritures
Service	Applique les règles métier, orchestre plusieurs repositories, notifie, audite	Du SQL brut, du HTTP
Repository	Construit et exécute les requêtes, applique le périmètre en SQL	Des décisions métier
Modèle	Décrit le schéma et les relations	De la logique applicative

Cette séparation rend chaque règle testable isolément et garantit qu'une nouvelle route ne peut pas contourner l'isolation multi-tenant : elle passerait forcément par un repository.

3. Gestion des transactions

Une requête HTTP correspond à une transaction :

1. `get_db()` ouvre une session (`AsyncSessionLocal`) pour la durée de la requête.
2. Les services enchaînent des `flush()` — les objets reçoivent leur identifiant et les contraintes sont vérifiées, sans validation définitive.
3. Le router appelle `session.commit()` une seule fois, en fin de traitement.
4. Toute exception déclenche un `rollback()` dans la dépendance.

Conséquence : une invitation d'agent qui échoue à l'étape de génération du jeton ne laisse jamais un `User` orphelin en base.

Pour le WebSocket et Celery — hors cycle HTTP — `session_scope()` fournit un contexte transactionnel équivalent avec commit automatique.

4. Isolation multi-tenant

Trois barrières successives :

a. Dépendance — `require_roles(...)` sur le router écarte les rôles non concernés.

b. Service — `ctx.require_same_agency(entity.agency_id)` sur les accès unitaires, plus les règles spécifiques (un agent n'accède qu'à son portefeuille).

c. Repository — `scoped_query(ctx)` traduit le périmètre en clauses SQL :

```
def scoped_query(self, ctx, *, agent_id=None, owner_id=None, public=False):
    stmt = select(Property).options(selectinload(Property.images))
    if public:
        return stmt.where(Property.is_published.is_(True), ...)
    if ctx.is_super_admin:
        return stmt
    if ctx.is_agency:
        return stmt.where(Property.agency_id == ctx.agency_id)
    if ctx.is_agent:
        return stmt.where(Property.agency_id == ctx.agency_id,
                          Property.agent_id == agent_id)
    if ctx.is_owner:
        return stmt.where(Property.owner_id == owner_id)
    ...
```

Un développeur qui ajoute un endpoint en oubliant la vérification métier obtient tout de même une liste filtrée : la fuite de données inter-agences est structurellement empêchée.

5. Le `SecurityContext`

Construit à chaque requête à partir du JWT puis enrichi par un accès base :

```python
@dataclass(slots=True)
class SecurityContext:
    user_id: UUID
    role: UserRole
    email: str
    agency_id: UUID | None      # tenant
    owner_id: UUID | None       # fiche propriétaire liée
    client_id: UUID | None      # fiche client liée
    permissions: set[str]       # dérivées du rôle
```

Le `Scope` y ajoute `agent_id` (résolu depuis la table `agents`), afin que les repositories filtrent sur l'identifiant métier et non sur l'identifiant utilisateur.

6. Temps réel

```
Client A ──ws──> Worker 1 ──┐
                            ├── Redis Pub/Sub (canal immoflow:events)
Client B ──ws──> Worker 3 ──┘
```

- `ConnectionManager` tient le registre des sockets **du worker courant**.
- `pubsub.publish(user_ids, payload)` publie l'événement sur Redis.
- `PubSubBridge`, lancée au démarrage, écoute le canal et relaie vers les sockets locales.
- Si Redis est indisponible, la diffusion retombe en local (mono-worker) et un avertissement est journalisé — le service ne s'interrompt pas.

Cette conception permet de multiplier les workers Uvicorn sans que deux utilisateurs connectés à des processus différents cessent de se voir.

7. Machines à états

Bien immobilier

```
DRAFT ──► AVAILABLE ──► RESERVED ──► SOLD ──► ARCHIVED
            ▲   │          │
            │   └──────────► RENTED ──► AVAILABLE
            └──────────────────┘
```

Les transitions autorisées sont déclarées dans `ALLOWED_TRANSITIONS` ; toute autre combinaison lève `BusinessRuleError` (HTTP 409).

- **Visite** : `REQUESTED → CONFIRMED | RESCHEDULED | REJECTED → COMPLETED | CANCELLED`
- **Vente** : `DRAFT → IN_PROGRESS → COMPLETED | CANCELLED` (une vente finalisée est figée)
- **Location** : `DRAFT → ACTIVE → TERMINATED | CANCELLED`
- **Propriétaire** : `INVITED → PENDING_VERIFICATION → ACTIVE | REJECTED | SUSPENDED`

- **Agence** : `PENDING_VALIDATION → ACTIVE ⇄ SUSPENDED | REJECTED`

Les transactions pilotent le statut du bien : finaliser une vente bascule le bien en `SOLD` et le dépublie ; résilier une location le remet en `AVAILABLE`.

8. Module IA — conception à double niveau

Chaque fonction IA suit le même schéma :

```
Requête
  ├─► Si un fournisseur LLM est configuré : appel + parsing JSON strict
  │      └─ échec réseau, quota, JSON invalide ──┐
  └─► Repli déterministe local  ◄────────────────┘
```

Le repli n'est pas un message d'erreur mais une implémentation complète :

- **Description** : gabarits français avec accord grammatical et élision.
- **Estimation** : médiane du prix au m² des biens comparables de la base (même quartier, surface ±40 %) ; à défaut, barème régional. Ajustements pour équipements et ancienneté, rendement locatif pour les locations, indice de confiance fonction du nombre de comparables.
- **Assistant** : extraction par expressions régulières (chambres, ville, budget en « millions », type de bien, équipements) branchée sur le moteur de recherche existant.
- **Analyse d'annonce** : contrôle de complétude sur le modèle `Property` + heuristiques de qualité rédactionnelle, score sur 100.

Bénéfice : aucune dépendance externe bloquante, coût maîtrisé, et les résultats de l'estimation s'appuient sur les **données réelles de l'agence** plutôt que sur les approximations d'un modèle de langage.

9. Observabilité

- Journaux JSON en production (`python-json-logger`), lisibles en développement.
- `X-Request-ID` propagé de bout en bout et repris dans les réponses d'erreur.
- `X-Process-Time-ms` sur chaque réponse.
- `AuditLog` : acteur, action, ressource, IP, user-agent, `request_id` , charge utile.
- Sondes `/health` (complète), `/health/live` (vivacité), `/health/ready` (disponibilité).

10. Choix techniques notables

DÉCISION	MOTIVATION
Horodatage côté Python (`default=_utcnow`) plutôt que `func.now()` seul	Évite l'expiration d'attribut après `UPDATE` et donc une E/S paresseuse pendant la sérialisation asynchrone ; `server_default` reste en place pour les insertions hors ORM
`NullPool` en environnement de test	Aucune connexion partagée entre boucles d'évènements
Jetons opaques hachés en SHA-256 pour invitations et réinitialisations	Une fuite de la base ne permet pas de rejouer un lien
Rotation systématique des refresh tokens	Le vol d'un jeton devient détectable et révoque la session
`selectinload` sur les médias des biens	Évite le problème N+1 sans jointure coûteuse
Filtre géographique par bounding box	Recherche de proximité utile sans imposer PostGIS en V1
Rate limiting fail-open	Une panne Redis dégrade la protection, jamais la disponibilité

11. Points d'extension

BESOIN	OÙ INTERVENIR
Nouveau fournisseur LLM	`services/ai_provider.py` — implémenter le protocole `LLMProvider`
Stockage S3	`services/storage.py` — implémenter le protocole `StorageBackend`
Nouveau rôle / permission	`core/permissions.py` + `scoped_query` des repositories concernés
Nouveau module métier	modèle → schéma → repository → service → router → migration
Recherche géographique fine	Activer PostGIS et remplacer le filtre par bounding box
Paiement, abonnement (V2)	Nouveau module isolé ; aucune dépendance à ajouter au cœur

PARTIE 3

Modules métier

Les douze modules, leurs règles, leurs statuts et leurs points d'entrée.

Ce document décrit, module par module, les règles métier appliquées par l'API, les statuts, les permissions et les endpoints associés.

Préfixe commun : `/api/v1`

1. Authentification

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
POST	`/auth/register`	Inscription (CLIENT, OWNER ou AGENCY)	Public
POST	`/auth/login`	Connexion	Public
POST	`/auth/refresh`	Renouvellement du jeton d'accès	Public (jeton requis)
POST	`/auth/logout`	Déconnexion (session ou tous appareils)	Authentifié
POST	`/auth/forgot-password`	Demande de réinitialisation	Public
POST	`/auth/reset-password`	Réinitialisation par jeton	Public
POST	`/auth/change-password`	Changement de mot de passe	Authentifié
POST	`/auth/accept-invitation`	Activation d'un compte invité	Public (jeton requis)
GET	`/auth/me`	Profil enrichi + permissions	Authentifié
PATCH	`/auth/me`	Mise à jour du profil	Authentifié
POST	`/auth/me/avatar`	Téléversement de l'avatar	Authentifié
GET	`/auth/sessions`	Sessions actives	Authentifié

Règles métier

- **Rôles auto-attribuables** : `CLIENT`, `OWNER`, `AGENCY` uniquement. Toute tentative d'inscription en `SUPER_ADMIN` ou `AGENT` est rejetée en **422** — ces rôles ne s'obtiennent que par invitation.
- **Politique de mot de passe** : 8 à 128 caractères, au moins une minuscule, une majuscule et un chiffre.
- **Inscription AGENCY** : `agency_name` obligatoire. L'agence est créée en `PENDING_VALIDATION` et l'utilisateur en `PENDING` — il peut se connecter mais son agence n'est pas opérationnelle avant validation par le Super Admin.
- **Rotation des refresh tokens** : chaque appel à `/auth/refresh` révoque le jeton utilisé et en émet un nouveau. Rejouer un jeton déjà consommé révoque **toutes** les sessions de l'utilisateur (signal de vol de jeton).
- **Non-énumération des comptes** : `/auth/login` et `/auth/forgot-password` répondent de façon identique que le compte existe ou non.
- **Changement de mot de passe** : révoque toutes les sessions actives.
- Les jetons de réinitialisation et d'invitation sont stockés **hachés (SHA-256)**. Ils ne sont jamais consultables en base en clair.

Réponse type

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer",
  "expires_at": "2026-07-31T19:05:00Z",
  "expires_in": 1800
}
```

---

2. Agences

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/agencies`	Lister les agences	SUPER_ADMIN
POST	`/agencies`	Créer une agence	SUPER_ADMIN
GET	`/agencies/me`	Mon agence	AGENCY, AGENT
GET	`/agencies/{id}`	Détail	Périmètre
PATCH	`/agencies/{id}`	Modifier	AGENCY, SUPER_ADMIN
POST	`/agencies/{id}/validate`	Valider ou rejeter	SUPER_ADMIN
POST	`/agencies/{id}/suspend`	Suspendre	SUPER_ADMIN
POST	`/agencies/{id}/activate`	Réactiver	SUPER_ADMIN
GET	`/agencies/{id}/stats`	Tableau de bord	Périmètre

Cycle de vie

```
PENDING_VALIDATION ──validate(approve=true)──► ACTIVE ⇄ SUSPENDED
        │
        └──validate(approve=false)──► REJECTED
```

Règles métier

- Seul le **Super Admin** valide, suspend ou réactive une agence.
- La validation active également le compte du dirigeant (`owner_user_id`) et lui envoie une notification + un email.
- Une agence `SUSPENDED` bloque la connexion de ses agents et de son dirigeant.

- Une agence non `ACTIVE` n'accepte pas les inscriptions autonomes de propriétaires.
- Le `slug` est généré automatiquement à partir du nom, avec suffixe numérique en cas de collision.
- Les documents légaux sont déposés via `/documents` avec `entity_type=AGENCY&category=AGENCY_LEGAL`.

Statistiques retournées

```
{
  "agents": 8, "owners": 42, "clients": 137, "properties": 96,
  "published_properties": 71, "visits_pending": 5,
  "sales_completed": 12, "rentals_active": 28
}
```

---

3. Agents

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/agents`	Lister	AGENCY, AGENT, SUPER_ADMIN
POST	`/agents`	Créer et inviter	AGENCY, SUPER_ADMIN
GET	`/agents/{id}`	Détail	Périmètre agence
PATCH	`/agents/{id}`	Modifier	AGENCY
POST	`/agents/{id}/resend-invitation`	Renvoyer l'invitation	AGENCY
POST	`/agents/{id}/suspend`	Suspendre	AGENCY
POST	`/agents/{id}/activate`	Activer	AGENCY
DELETE	`/agents/{id}`	Supprimer	AGENCY

Workflow d'invitation

```
1. L'agence crée l'agent (email, nom, téléphone, fonction)
   → un User de rôle AGENT est créé SANS mot de passe, statut PENDING
   → un profil Agent est créé, statut INVITED
2. Un jeton d'invitation (48 h) est généré et envoyé par email
3. L'agent appelle POST /auth/accept-invitation { token, password }
   → User.status = ACTIVE, Agent.status = ACTIVE, activated_at renseigné
```

En développement (`ENVIRONMENT ≠ production`), la réponse contient `invitation_url` pour faciliter les tests. En production, ce champ est `null` — le lien passe uniquement par email.

Permissions fines

CHAMP	EFFET
`can_manage_owners`	Autorise la création et le suivi de propriétaires
`can_publish_properties`	Autorise la publication au catalogue

Périmètre d'un agent

Un agent ne voit que ce qui lui est **attribué** :

RESSOURCE	CRITÈRE DE RATTACHEMENT
Biens	`Property.agent_id == agent.id`
Clients	Lien actif dans `client_agents`
Propriétaires	`Owner.agent_id == agent.id`
Visites	`Visit.agent_id == agent.id`
Transactions	`Sale.agent_id` / `Rental.agent_id == agent.id`

4. Propriétaires

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
POST	`/owners/register`	Inscription autonome (workflow 2)	Public
POST	`/owners/accept-invitation`	Accepter une invitation (workflow 1)	Public
GET	`/owners`	Lister	Périmètre
POST	`/owners`	Créer et inviter	AGENCY, AGENT
GET	`/owners/{id}`	Détail	Périmètre
PATCH	`/owners/{id}`	Modifier	Périmètre
POST	`/owners/{id}/complete-profile`	Compléter le profil	Propriétaire, agence
POST	`/owners/{id}/verify`	Vérifier le dossier	AGENCY
POST	`/owners/{id}/suspend`	Suspendre	AGENCY

Workflow 1 — invitation par l'agent

```
Agent crée le propriétaire (nom, prénom, email, téléphone)
   → Owner status = INVITED, un User de rôle OWNER est pré-créé
   → invitation (72 h) envoyée par email
Propriétaire accepte : POST /owners/accept-invitation { token, password }
   → status = PENDING_VERIFICATION
Propriétaire complète son profil + dépose sa CIP
   → POST /owners/{id}/complete-profile
   → POST /documents (entity_type=OWNER, category=IDENTITY_CIP)
Agence vérifie : POST /owners/{id}/verify { approve: true }
   → status = ACTIVE
```

Workflow 2 — inscription autonome

```
POST /owners/register { agency_id, nom, prénom, email, téléphone, password }
   → l'agence doit être ACTIVE, sinon 409
   → Owner status = PENDING_VERIFICATION
   → notification envoyée au dirigeant de l'agence
Agence vérifie → ACTIVE
```

Statuts

STATUT	SIGNIFICATION
`INVITED`	Créé par l'agence, invitation non acceptée
`PENDING_VERIFICATION`	Compte actif, dossier en attente de contrôle
`ACTIVE`	Dossier vérifié, peut détenir des biens
`REJECTED`	Dossier refusé (motif conservé)
`SUSPENDED`	Accès suspendu par l'agence

Règle bloquante

`POST /owners/{id}/verify` avec `approve: true` renvoie **409** `business_rule_violation` si aucun document de catégorie `IDENTITY_CIP` n'a été déposé pour ce propriétaire. La conformité documentaire est donc garantie par l'API, pas seulement par la discipline des utilisateurs.

Unicité

Un couple `(agency_id, email)` est unique : un même propriétaire peut être suivi par plusieurs agences, mais jamais dupliqué au sein d'une même agence.

5. Biens immobiliers

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/properties/types`	Types de biens	Authentifié
POST	`/properties/types`	Créer un type	AGENCY, AGENT
GET	`/properties`	Rechercher (périmètre RBAC)	Authentifié
POST	`/properties`	Créer	AGENCY, AGENT
GET	`/properties/{id}`	Détail	Périmètre
PATCH	`/properties/{id}`	Modifier	AGENCY, AGENT
DELETE	`/properties/{id}`	Supprimer	AGENCY, AGENT
POST	`/properties/{id}/status`	Changer le statut	AGENCY, AGENT
POST	`/properties/{id}/publish`	Publier / dépublier	AGENCY, AGENT
GET	`/properties/{id}/images`	Médias	Périmètre
POST	`/properties/{id}/images`	Ajouter par URL	AGENCY, AGENT
POST	`/properties/{id}/images/upload`	Téléverser	AGENCY, AGENT
DELETE	`/properties/{id}/images/{img}`	Supprimer un média	AGENCY, AGENT
GET	`/public/properties`	Catalogue public	**Public**
GET	`/public/properties/{id}`	Détail public	**Public**

Champs

Titre, description, type d'opération (`SALE` / `RENT` / `BOTH`), type de bien, adresse, quartier, ville, pays, latitude, longitude, prix, devise, charges, caution, surface, surface du terrain, pièces, chambres, salles d'eau, étage, année de construction, équipements (tableau JSONB), photos et vidéos.

Chaque bien reçoit une **référence lisible** générée automatiquement : `BIEN-<4 car. agence>-<numéro>`, par exemple `BIEN-9F80-00001`.

Transitions de statut



Une transition non autorisée (par exemple `DRAFT → SOLD`) renvoie **409** :

```
{ "error": { "code": "business_rule_violation",
             "message": "Transition de statut interdite : DRAFT → SOLD." } }
```

Conditions de publication

`POST /properties/{id}/publish` vérifie la présence de :

- un titre, un prix, une ville ;
- une description d'au moins 40 caractères ;
- au moins une photo.

En cas de manque, réponse **409** avec le détail :

```
{ "error": { "code": "business_rule_violation",
             "message": "Publication impossible : éléments manquants.",
             "details": { "missing": ["description (40 caractères minimum)", "au moins une
photo"] } } }
```

Publier un bien en `DRAFT` le bascule automatiquement en `AVAILABLE`. Le passage en `SOLD`, `RENTED` ou `ARCHIVED` le dépublie automatiquement.

Suppression

Un bien `SOLD` ou `RENTED` **ne peut pas être supprimé** (traçabilité des transactions) : il doit être archivé.

Filtres de recherche

`q`, `city`, `district`, `listing_type`, `status`, `property_type_id`, `owner_id`, `agent_id`, `min_price`, `max_price`, `min_area`, `max_area`, `bedrooms`, `rooms`, `is_published`, `near_lat` + `near_lng` + `radius_km`, `sort` (`price`, `-price`, `-created_at`, `area`, `views_count`), `page`, `size`.

Exemple :

```
GET /api/v1/properties?city=Cotonou&min_price=20000000&max_price=80000000
    &bedrooms=3&sort=-created_at&page=1&size=20
```

Catalogue public

`/public/properties` n'expose que les biens `is_published = true` avec statut `AVAILABLE` ou `RESERVED`. Aucun jeton requis. Le compteur de vues est incrémenté à chaque consultation de fiche.

---

6. CRM Clients

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/clients`	Lister	Périmètre
POST	`/clients`	Créer	AGENCY, AGENT
GET	`/clients/{id}`	Détail	Périmètre
PATCH	`/clients/{id}`	Modifier	Périmètre
DELETE	`/clients/{id}`	Supprimer	AGENCY
POST	`/clients/{id}/assign`	Attribuer à un agent	AGENCY, AGENT
DELETE	`/clients/{id}/assign/{agent}`	Retirer l'attribution	AGENCY
GET	`/clients/{id}/favorites`	Favoris	Périmètre
POST	`/clients/{id}/favorites/{bien}`	Ajouter un favori	Périmètre
DELETE	`/clients/{id}/favorites/{bien}`	Retirer un favori	Périmètre
GET	`/clients/{id}/search-history`	Historique de recherche	Périmètre
POST	`/clients/{id}/requests`	Créer une demande	Périmètre
GET	`/clients/requests/all`	Demandes de l'agence	AGENCY, AGENT

Règle centrale

> **Un client appartient à un agent après interaction.**

Le rattachement est matérialisé par la table `client_agents` :

- création par un agent → attribution automatique à cet agent, marqué `is_primary` ;
- un client peut être suivi par plusieurs agents, un seul étant principal ;
- le retrait d'attribution ne supprime pas la ligne : il renseigne `unassigned_at` (traçabilité).

Conséquence sur le périmètre : un agent ne voit que les clients dont il détient une attribution active. Le dirigeant d'agence voit tous les clients de son agence.

Données suivies

Coordonnées, ville, source d'acquisition, notes, budget minimum et maximum, préférences (JSONB libre), favoris, historique de recherche (alimenté automatiquement par l'assistant IA), demandes.

---

7. Visites

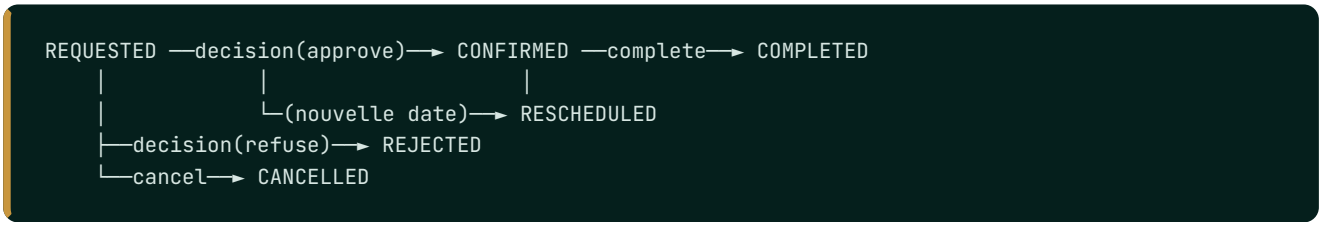
Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/visits`	Lister	Périmètre
POST	`/visits`	Demander ou programmer	Authentifié
GET	`/visits/{id}`	Détail	Périmètre
POST	`/visits/{id}/decision`	Valider / refuser / reprogrammer	AGENCY, AGENT
PATCH	`/visits/{id}`	Modifier	AGENCY, AGENT
POST	`/visits/{id}/cancel`	Annuler	Parties prenantes
POST	`/visits/{id}/complete`	Clôturer	AGENCY, AGENT

Comportement selon l'auteur

AUTEUR	STATUT INITIAL	REMARQUE
CLIENT	`REQUESTED`	Le bien doit être publié
AGENT / AGENCY	`CONFIRMED`	`client_id` obligatoire dans la requête

Cycle de vie



Règles métier

- La date de visite doit être **dans le futur** (409 sinon).
- Un bien `SOLD` ou `RENTED` n'accepte plus de demande de visite.
- Une visite `COMPLETED` ou `CANCELLED` ne peut plus être modifiée ni reclôturée.
- L'agent est déduit automatiquement : agent du bien, à défaut agent principal du client.
- Toute modification notifie **client, agent et propriétaire** (sauf l'auteur de l'action) via notification in-app + WebSocket.
- La clôture accepte un retour (`feedback`) et une note de 1 à 5.

---

8. Transactions

Ventes

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/sales`	Lister	Périmètre
POST	`/sales`	Créer	AGENCY, AGENT
GET	`/sales/{id}`	Détail	Périmètre
PATCH	`/sales/{id}`	Modifier	AGENCY, AGENT

Associe **client + bien + agent + propriétaire**. Référence `VTE—<agence>-<n°>` .

- La commission est calculée automatiquement : `montant × taux / 100` , arrondie au centime.
- À la création, le bien passe en `RESERVED` .
- `status = COMPLETED` → le bien passe en `SOLD` et est dépublié ; `completed_at` et `signed_at` sont horodatés.
- `status = CANCELLED` → le bien redevient `AVAILABLE` s'il était réservé.
- Une vente `COMPLETED` est **figée** : tout changement de statut ultérieur est refusé.

Locations

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/rentals`	Lister	Périmètre
POST	`/rentals`	Créer	AGENCY, AGENT
GET	`/rentals/{id}`	Détail	Périmètre
PATCH	`/rentals/{id}`	Modifier	AGENCY, AGENT

Loyer mensuel, charges, caution, devise, date de début, date de fin, durée en mois. Référence `LOC-<agence>-<n°>`.

- `status = ACTIVE` → le bien passe en `RENTED` et est dépublié.
- `status = TERMINATED` → le bien redevient `AVAILABLE`, `terminated_at` horodaté.
- La date de fin doit être postérieure à la date de début (validation Pydantic).

Contrats

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/contracts`	Lister	AGENCY, AGENT
POST	`/contracts`	Créer	AGENCY, AGENT
PATCH	`/contracts/{id}`	Modifier	AGENCY, AGENT

Types : `SALE_DEED` (acte de vente), `LEASE` (bail), `MANDATE` (mandat). Statuts : `DRAFT → PENDING_SIGNATURE → SIGNED → TERMINATED`. Un contrat doit être lié à une vente, une location ou un bien, et peut référencer un document signé (`document_id`).

---

9. Documents

Endpoints

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/documents`	Lister	Périmètre
POST	`/documents`	Téléverser	Authentifié
GET	`/documents/{id}`	Métadonnées	Périmètre
GET	`/documents/{id}/download`	Télécharger	Périmètre
POST	`/documents/{id}/verify`	Valider / rejeter	AGENCY
DELETE	`/documents/{id}`	Supprimer	Déposant ou agence

Rattachement polymorphe contrôlé

`entity_type` ∈ `USER`, `AGENCY`, `OWNER`, `CLIENT`, `PROPERTY`, `CONTRACT` `entity_id` = identifiant de l'entité concernée.

L'accès à l'entité est vérifié **avant** l'enregistrement : un propriétaire ne peut déposer un document que sur sa propre fiche ou ses propres biens.

Catégories

`IDENTITY_CIP` (pièce d'identité), `CONTRACT`, `MANDATE`, `PROPERTY_TITLE`, `AGENCY_LEGAL`, `OTHER`.

Sécurité des fichiers

- Liste blanche MIME : images (JPEG, PNG, WebP, GIF, HEIC), documents (PDF, Word, Excel, texte), vidéos (MP4, QuickTime, WebM).
- Taille maximale configurable (`MAX_UPLOAD_SIZE_MB`, 25 Mo par défaut).
- Nom de fichier assaini, clé de stockage préfixée d'un UUID (aucune collision).
- Protection contre la traversée de chemin (`path traversal`).
- Empreinte SHA-256 calculée et stockée.
- **Les fichiers ne sont jamais servis en accès direct** : le téléchargement passe par `/documents/{id}/download`, qui applique le contrôle d'accès et journalise l'action.

Vérification

`POST /documents/{id}/verify` (agence uniquement) fait passer le document en `VERIFIED` ou `REJECTED` avec motif, et notifie le déposant.

---

10. Messagerie temps réel

Endpoints REST

MÉTHODE	CHEMIN	DESCRIPTION
GET	`/conversations`	Mes conversations (avec compteur de non-lus)
POST	`/conversations`	Créer ou retrouver une conversation
GET	`/conversations/{id}`	Détail
GET	`/conversations/{id}/messages`	Historique paginé
POST	`/conversations/{id}/messages`	Envoyer un message
POST	`/conversations/{id}/read`	Marquer comme lu
POST	`/conversations/{id}/archive`	Archiver

WebSocket

```
ws://<host>/ws?token=<access_token>
```

SENS	ÉVÉNEMENTS
Entrants	`message.send`, `message.read`, `typing`, `ping`
Sortants	`connected`, `message.new`, `message.read`, `typing`, `notification.new`, `pong`, `error`

Types de conversation

`CLIENT_AGENT`, `OWNER_AGENT`, `AGENT_AGENCY`, `SUPPORT`.

Types de message

`TEXT`, `IMAGE`, `DOCUMENT`, `VIDEO`, `SYSTEM`. Un message texte exige un contenu non vide ; un message média exige `attachment_url` ou `document_id` (validation Pydantic).

Règles métier

- Tous les participants doivent appartenir au **même tenant** — la création d'une conversation inter-agences est refusée (403).
- La déduplication est automatique : recréer une conversation entre les mêmes participants et du même type retourne la conversation existante.
- Une conversation archivée n'accepte plus de message.
- Chaque message envoyé est **persisté**, **diffusé** aux participants connectés (via Redis Pub/Sub, donc sur tous les workers) et génère une **notification** pour les destinataires.
- Le compteur de non-lus se calcule à partir de `last_read_at` du participant.
- L'historique se pagine par curseur temporel (`before`) pour un défilement infini stable.

---

11. Notifications

MÉTHODE	CHEMIN	DESCRIPTION
GET	`/notifications`	Mes notifications (filtres `is_read`, `type`)
GET	`/notifications/unread-count`	Nombre de non lues
POST	`/notifications/read`	Marquer comme lues (`ids` ou `all: true`)

Types

`INVITATION`, `ACCOUNT_VALIDATION`, `NEW_MESSAGE`, `VISIT`, `TRANSACTION`, `DOCUMENT`, `SYSTEM`.

Canaux

1. **In-app** — persistée en base, consultable via l'API ;

2. **WebSocket** — poussée immédiate (`notification.new`) si l'utilisateur est connecté ;
3. **Email** — optionnel, activé au cas par cas (validation de compte, invitation).

Chaque notification porte un `action_url` exploitable par le frontend et une charge utile `payload` (identifiants de la ressource concernée).

---

12. IA immobilière

MÉTHODE	CHEMIN	DESCRIPTION
POST	`/ai/description`	Génération de description d'annonce
POST	`/ai/estimate`	Estimation de prix
POST	`/ai/assistant`	Recherche en langage naturel + conseil
POST	`/ai/analyze-listing`	Analyse de qualité d'annonce

Permission requise : `ai:use` (SUPER_ADMIN, AGENCY, AGENT, CLIENT).

12.1 Génération de description

Entrée : `property_id` (les caractéristiques sont alors chargées depuis le bien) ou saisie manuelle (type, surface, quartier, chambres, équipements, ton, longueur).

Sortie

```
{
  "description": "Découvrez cette villa à vendre à Fidjrossè, Cotonou. Elle offre une surface de
320 m², 4 chambres…",
  "title_suggestion": "Villa 4 chambres — 320 m² à Fidjrossè, Cotonou",
  "highlights": ["Piscine", "Garage", "Jardin"],
  "provider": "local"
}
```

12.2 Estimation de prix

Méthode :

1. Recherche des **biens comparables** en base : même quartier (à défaut même ville), surface comprise entre 60 % et 140 % de la surface cible.
2. Si au moins 3 comparables : **médiane du prix au m²** (méthode privilégiée).
3. Sinon : barème régional au m² (Cotonou, Calavi, Porto-Novo, Parakou, Bohicon…).
4. Ajustements : équipements (piscine +12 %, vue mer +15 %, meublé +8 %…, plafonné à +35 %), ancienneté (−0,5 % par an, plafonné à −20 %).

5. Pour une location : application d'un rendement locatif mensuel.
6. Indice de confiance : 0,75 (≥ 5 comparables), 0,60 (≥ 3), 0,40 (barème).
7. Fourchette : ±12 % si confiance élevée, ±22 % sinon.

Sortie


```
{
  "suggested_price": 144000000,
  "price_range_min": 126720000,
  "price_range_max": 161280000,
  "currency": "XOF",
  "confidence": 0.6,
  "explanation": "Estimation basée sur la médiane de 4 biens comparables, soit environ 450 000 XOF/m² pour 320 m². Ajustement des équipements et de l'ancienneté : +21 %. Fourchette de confiance ±12 %.",
  "comparables_count": 4,
  "method": "médiane de 4 biens comparables",
  "provider": "local"
}
```


12.3 Assistant immobilier

Entrée : `"Maison 4 chambres proche centre budget 60 millions"`

Critères extraits


```
{ "listing_type": null, "property_type": "maison", "city": "Cotonou",
  "bedrooms": 4, "max_price": 60000000, "amenities": [] }
```


L'extraction reconnaît les nombres en chiffres et en lettres, les unités de budget (« millions », « milliards », « k »), les villes du Bénin, les types de biens et les équipements. Les critères alimentent directement le moteur de recherche, **dans le périmètre RBAC de l'appelant** (un client ne voit que le catalogue publié).

Pour un client, chaque requête alimente automatiquement son **historique de recherche**.

12.4 Analyse d'annonce

Contrôle de complétude (photos, surface, chambres, pièces, adresse, quartier, ville, géolocalisation, propriétaire rattaché, équipements) et de qualité rédactionnelle (longueur, majuscules, ponctuation excessive, numéro de téléphone dans le texte, titre trop court pour le référencement).

Sortie : score sur 100, `missing_fields`, `quality_issues`, `suggestions`.

Fournisseurs LLM

`AI_PROVIDER` ∈ `none` | `openai` | `groq` | `anthropic`, avec `AI_API_KEY` et `AI_MODEL`.

> **Sans clé configurée, les quatre fonctions restent pleinement opérationnelles** grâce aux replis déterministes décrits ci-dessus. Si un appel LLM échoue (réseau, quota, JSON invalide), l'API bascule silencieusement sur le repli : aucune fonctionnalité ne devient indisponible à cause d'un service tiers.

---

13. Administration et audit

MÉTHODE	CHEMIN	DESCRIPTION	ACCÈS
GET	`/admin/stats`	Statistiques plateforme	SUPER_ADMIN
GET	`/admin/audit-logs`	Journal d'audit	AGENCY, SUPER_ADMIN
GET	`/users`	Lister les utilisateurs	AGENCY, SUPER_ADMIN
GET	`/users/{id}`	Détail	Périmètre
PATCH	`/users/{id}`	Mise à jour administrative	AGENCY, SUPER_ADMIN
POST	`/users/{id}/deactivate`	Désactiver un compte	AGENCY, SUPER_ADMIN

Journal d'audit

Chaque action sensible enregistre : acteur, agence, action, type et identifiant de ressource, statut, adresse IP, user-agent, `request_id` et charge utile.

Actions journalisées : `auth.register`, `auth.login`, `auth.change_password`, `auth.reset_password`, `auth.accept_invitation`, `agency.create`, `agency.validate`, `agency.suspend`, `agency.activate`, `agent.invite`, `agent.update`, `agent.set_status`, `agent.delete`, `owner.invite`, `owner.self_register`, `owner.verify`, `owner.suspend`, `client.create`, `client.assign`, `client.delete`, `property.create`, `property.update`, `property.change_status`, `property.publish`, `property.delete`, `visit.request`, `visit.decide`, `sale.create`, `rental.create`, `contract.create`, `document.upload`, `document.download`, `document.verify`, `document.delete`, `user.admin_update`, `user.deactivate`.

Le journal est **filtré par agence** : un dirigeant ne consulte que les actions de son tenant. L'écriture d'audit n'interrompt jamais le flux métier — un échec est journalisé mais n'annule pas l'opération.

Restrictions

Un compte ne peut pas se désactiver lui-même. Seul le Super Admin peut modifier le rôle ou l'agence de rattachement d'un utilisateur, et l'attribution du rôle `SUPER_ADMIN` lui est réservée.

---

14. Matrice des permissions

PERMISSION	SUPER_ADMIN	AGENCY	AGENT	OWNER	CLIENT
`agency:read`	✓	✓	✓	—	—
`agency:update`	✓	✓	—	—	—
`agent:create` / `update` / `delete`	✓	✓	—	—	—
`owner:create`	✓	✓	✓	—	—
`owner:read`	✓	✓			—
`owner:verify`	✓	✓	—	—	—
`client:create` / `update`	✓	✓	✓	—	✓ [1]
`client:delete`	✓	✓	—	—	—
`property:create` / `update`	✓	✓		—	—
`property:read`	✓				✓ [2]
`property:publish`	✓	✓	✓	—	—
`property:delete`	✓	✓	—	—	—
`visit:create`	✓	—	—	—	
`visit:read`	✓	✓	✓	✓	✓
`visit:update`	✓	✓	✓	—	—
`transaction:create` / `update`	✓	✓		—	—
`transaction:read`	✓	✓	✓	✓	✓
`document:create`	✓	✓	✓	✓	—
`document:verify` / `delete`	✓	✓	—	—	—
`conversation:create` / `message:create`	✓	✓			
`notification:read`					
`ai:use`				—	
`audit:read`			—	—	—

[1] Uniquement sa propre fiche client. [2] Catalogue publié uniquement.

Toutes ces permissions s'appliquent **en plus** du filtrage par périmètre décrit dans chaque module : disposer de `property:read` ne donne accès qu'aux biens de son propre périmètre, jamais à ceux d'une autre agence.

PARTIE 4

Jeu de démonstration

Peupler la base avec des données cohérentes et vérifiées.

Le script `backend/scripts/seed_data.py` remplit la base avec un jeu de données cohérent : agences, équipes, propriétaires, clients, biens photographiés, visites, transactions, documents, conversations, notifications et journal d'audit.

Utilisation

Depuis `backend/`, sur une base déjà migrée :

```
createdb immoflow                       # si nécessaire
alembic upgrade head
python -m app.database.init_db          # Super Admin + types de biens

PYTHONPATH=. python scripts/seed_data.py
```

Avec Docker :

```
docker compose exec api python scripts/seed_data.py
```

Volumes

Valeurs par défaut, toutes ajustables :

OPTION	DÉFAUT	EFFET
`--agencies`	4	Agences créées
`--agents`	4	Agents dans l'agence principale (moitié pour les autres)
`--owners`	8	Propriétaires dans l'agence principale
`--clients`	12	Clients dans l'agence principale
`--properties`	30	Biens, répartis entre les agences actives
`--visits`	45	Visites, passées et à venir
`--sales`	20 % des biens	Ventes générées
`--rentals`	15 % des biens	Locations générées
`--conversations`	4	Conversations par agence
`--audit-logs`	40	Entrées d'audit par agence
`--password`	`Demo2026!`	Mot de passe des comptes créés

Exemples :

```
# Jeu de démonstration standard
PYTHONPATH=. python scripts/seed_data.py

# Base fournie pour des tests de charge
PYTHONPATH=. python scripts/seed_data.py --agencies 6 --properties 200 \
    --clients 40 --owners 25 --agents 6 --visits 300

# Repartir de zéro
PYTHONPATH=. python scripts/seed_data.py --reset --properties 50

# Vider sans recréer
PYTHONPATH=. python scripts/seed_data.py --reset-only
```

Générer 200 biens et l'ensemble de leurs dépendances prend environ 4 secondes.

Comptes créés

Un compte par rôle est garanti sur la première agence, avec le même mot de passe :

RÔLE	IDENTIFIANT	MOT DE PASSE
Super Admin	`admin@immoflow.ai`	celui défini par `init_db` (`ChangeMe123!` par défaut)
Agence	`agence@immoflow.ai`	`Demo2026!`
Agent	`agent@immoflow.ai`	`Demo2026!`
Propriétaire	`proprietaire@immoflow.ai`	`Demo2026!`
Client	`client@immoflow.ai`	`Demo2026!`

Les autres comptes générés utilisent le même mot de passe et des adresses dérivées des noms.

Ce que le script garantit

Le réalisme d'un jeu de données ne tient pas au volume mais à la cohérence. Le script respecte les mêmes règles que l'API :

- chaque bien appartient à un propriétaire **de la même agence**, jamais d'une autre ;
- aucun bien n'est rattaché à un propriétaire resté au stade « invité » — l'API le refuserait ;
- un bien publié possède toujours une photo de couverture ;
- un bien vendu ou loué est retiré du catalogue public ;
- aucun brouillon n'est publié ;
- les visites et les transactions portent sur des biens de leur propre agence ;
- une vente finalisée place le bien en « vendu », un bail actif le place en « loué ».

Sept contrôles sont exécutés en fin de génération et affichés. Si l'un échoue, le script s'arrête avec un code de sortie non nul — utilisable en intégration continue.

Le script garantit également qu'**aucun écran du produit n'est vide** : chaque compte de démonstration dispose de biens, de visites, de transactions, de documents, de conversations et de notifications.

Reproductibilité

La graine aléatoire est fixe (`SEED = 20260803`). Deux exécutions avec les mêmes options produisent le même jeu de données, ce qui rend les captures d'écran et les démonstrations stables.

Suppression

`--reset` et `--reset-only` ne suppriment **que** les données générées. Les agences créées par le script portent un marqueur (`settings.seed = "true"`) et la suppression est limitée à leur périmètre.

Concrètement :

- le compte Super Admin est préservé ;
- les types de biens créés par `init_db` sont préservés ;
- toute agence, tout bien, tout compte créé manuellement est préservé.

Répartition des statuts

Le jeu de données couvre volontairement tous les cas que le produit doit savoir afficher, pas seulement le cas nominal :

ENTITÉ	STATUTS REPRÉSENTÉS
Agences	actives, une en attente de validation, une suspendue
Agents	actifs, un invité par agence (compte non encore activé)
Propriétaires	vérifiés (majorité), en attente de vérification, invités
Biens	disponibles, réservés, vendus, loués, brouillons, archivés
Visites	demandées, confirmées, reprogrammées, effectuées, annulées, refusées, absences
Ventes	en cours et finalisées
Locations	actives et terminées
Documents	validés, en attente, avec pièces d'identité et titres fonciers

C'est ce qui permet de démontrer l'écran de validation des agences, la vérification des dossiers propriétaires ou le traitement des demandes de visite : sans données dans ces états, ces écrans resteraient vides.

PARTIE 5

Installation

Mise en route locale, avec ou sans Docker.

Option 1 — Docker (recommandé)

Prérequis : Docker 24+, Docker Compose v2.

```
git clone <repo> && cd immoflow-ai-api
cp backend/.env.example backend/.env
```

Éditez `backend/.env` et renseignez au minimum :

```
SECRET_KEY=<sortie de: openssl rand -hex 32>
FIRST_SUPERADMIN_EMAIL=votre.admin@domaine.com
FIRST_SUPERADMIN_PASSWORD=<mot de passe fort>
```

```
docker compose up --build
```

L'entrypoint attend PostgreSQL, applique les migrations, crée le Super Admin et les types de biens, puis démarre l'API sur http://localhost:8000/docs.

Commandes utiles :

```
docker compose logs -f api          # journaux
docker compose exec api bash        # shell dans le conteneur
docker compose exec db psql -U immoflow -d immoflow
docker compose down                 # arrêt
docker compose down -v              # arrêt + purge des volumes
```

Option 2 — Installation locale

Prérequis : Python 3.12+, PostgreSQL 16+, Redis 7+ (facultatif).

1. Environnement Python

```
cd backend
python -m venv .venv
source .venv/bin/activate            # Windows : .venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
```

2. Base de données

```
sudo -u postgres psql -c "CREATE USER immoflow WITH PASSWORD 'immoflow';"
sudo -u postgres psql -c "CREATE DATABASE immoflow OWNER immoflow;"
```

3. Configuration

```
cp .env.example .env
```

Vérifiez `POSTGRES_*`, `REDIS_*` et générez `SECRET_KEY`.

4. Migrations et données de référence

```
alembic upgrade head
python -m app.database.init_db
```

5. Démarrage

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Vérification de l'installation

```
curl http://localhost:8000/health
# {"status":"healthy","database":"up","redis":"up","version":"1.0.0"}

curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@immoflow.ai","password":"ChangeMe123!"}'
```

Dépannage

SYMPTÔME	CAUSE PROBABLE	SOLUTION
`Base de données indisponible` au démarrage	PostgreSQL arrêté ou identifiants erronés	`pg_isready`, vérifiez `POSTGRES_*` dans `.env`
`Redis indisponible`	Redis absent	Sans effet bloquant : quotas et diffusion multi-workers désactivés
`alembic: command not found`	Environnement virtuel non activé	`source .venv/bin/activate`
401 sur toutes les routes	Jeton expiré	`POST /api/v1/auth/refresh`
403 `permission_denied`	Rôle insuffisant ou ressource hors périmètre	Vérifiez le rôle via `GET /api/v1/auth/me`
409 sur la publication d'un bien	Éléments obligatoires manquants	Le champ `details.missing` liste ce qui manque
WebSocket fermé immédiatement	Jeton absent ou invalide	`ws://…/ws?token=<access_token>`
Erreur de migration `type already exists`	Base partiellement migrée	`alembic downgrade base` puis `alembic upgrade head`

Mise à jour

```
git pull
pip install -r backend/requirements.txt   # nouvelles dépendances
cd backend && alembic upgrade head        # nouvelles migrations
```

En Docker : `docker compose up --build -d` (les migrations sont appliquées automatiquement).